OpenTelemetry

instrument once, send anywhere

Watch the 5-min video

In one line: OpenTelemetry is the vendor-neutral standard for telemetry: one set of APIs, SDKs and semantic conventions for traces, metrics and logs, one wire protocol (OTLP), and a Collector that receives, processes and exports data, so the backend becomes a config choice, not a code change.

1 Three vendors, three agents

Your team uses one vendor for logs, another for metrics and a third for traces. Each needs its own agent and library wired into every service. Switching vendors means re-instrumenting everything, so nobody switches. OpenTelemetry (OTel, a CNCF project formed in 2019 from the merger of OpenTracing and OpenCensus) fixes that: instrument once, send anywhere.

2 What OpenTelemetry standardizes

- **API**: what your code and libraries call (start a span, record a metric). Safe to depend on; a no-op without an SDK.
- **SDK**: the implementation per language: sampling, batching, exporting.
- **OTLP**: one protocol for all signals, over gRPC (port 4317) or HTTP (port 4318).
- **Semantic conventions**: standard attribute names, so data looks the same from every language.
- **Collector**: a separate process that receives, processes and exports telemetry.

It is supported by every major vendor and cloud. OTel does not store or display data: that is still the job of a backend.

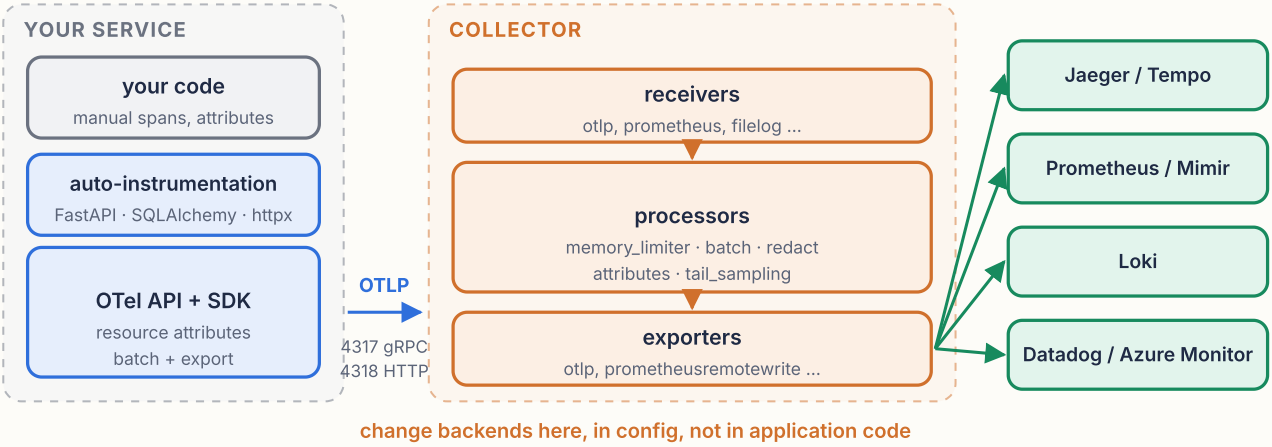


Figure 1. The SDK exports OTLP to a Collector; the Collector's pipeline of receivers, processors and exporters fans out to any backend.

3 Auto vs manual instrumentation

	Auto-instrumentation	Manual instrumentation
How	wrappers around popular libraries, no code changes	API calls in your code
Gives you	server spans, DB queries, HTTP clients, queues; propagation	business steps ("pricing"), domain attributes (plan, tenant)
Effort	install packages, start with a wrapper or agent	a few lines where it matters

Start with auto; it gets you surprisingly far. Then add manual spans and attributes where the questions are about your domain.

4 The Collector

Receivers accept OTLP and other formats (Prometheus scrape, Jaeger, log files). **Processors** batch, limit memory, drop noisy attributes, **redact secrets** and **sample traces** (tail sampling, lesson 63). **Exporters** send to Jaeger, Tempo, Prometheus, Loki, Datadog, Azure Monitor, often several at once. Run it as an **agent** on each node (low-latency hand-off, adds host metadata), as a central **gateway** (sampling, keys and routing in one place), or both.

5 Semantic conventions and resources

Semantic conventions fix names such as `service.name`, `http.request.method`, `http.response.status_code` and `db.system`. Every library in every language uses them, so one dashboard or alert works across Python, Java and Go services. **Resource attributes** describe the emitter: always set `service.name`, `service.version` and the environment, or everything shows up as `unknown_service`. Upgrade SDKs regularly: the project and its conventions move fast.

```
pip install opentelemetry-distro opentelemetry-exporter-otlp
opentelemetry-bootstrap -a install          # adds fastapi, sqlalchemy, httpx instrumentations
OTEL_SERVICE_NAME=orders \
OTEL_RESOURCE_ATTRIBUTES=service.version=2.4.0,deployment.environment=prod \
OTEL_EXPORTER_OTLP_ENDPOINT=http://collector:4317 \
opentelemetry-instrument uvicorn app:app

# one manual span where only you know what matters
tracer = trace.get_tracer(__name__)
with tracer.start_as_current_span("pricing") as span:
    span.set_attribute("customer.plan", plan)
    price = compute_price(cart, plan)
```

Figure 2. The video's FastAPI + SQLAlchemy + httpx setup: zero-code instrumentation via environment variables, plus one manual span.

WORKED EXAMPLE: INSTRUMENTING A FASTAPI SERVICE

Install the distro and the instrumentation packages, start the app under `opentelemetry-instrument`, point OTLP at the collector (Figure 2). Every request now becomes a trace: a server span for `POST /orders`, a child span per SQL query, and a client span per outgoing httpx call, with `traceparent` passed to the next service. One manual span wraps the pricing logic and tags the customer's plan. There is no vendor code in the app; switching from Jaeger to Azure Monitor is a collector config change.

TRADE-OFFS

- **Collector vs direct export:** a collector adds a hop and something to run, but centralises sampling, redaction and backend keys.
- **Auto-instrumentation** is quick but generic and adds some overhead; manual spans cost code but carry business meaning.
- **Maturity varies** by signal and language: traces are the most mature; check the status of logs and metrics for your SDK.
- **Vendor features** beyond OTel (profilers, special agents) can bring back lock-in: use them deliberately.

6 Key terms

OpenTelemetry (OTel): open standard and tooling for traces, metrics, logs

Collector: receives, processes and exports telemetry

Resource: attributes of the emitter, e.g. service.name

OTLP: OTel's wire protocol (gRPC 4317, HTTP 4318)

Semantic conventions: standard attribute names

Auto-instrumentation: library wrappers that create spans without code changes

7 Interview questions

Q1. What problem does OpenTelemetry solve? BASIC

It replaces vendor-specific agents and libraries with one open standard (APIs, SDKs, OTLP) for traces, metrics and logs, so you instrument once and send the data to any backend.

Q2. What does the Collector do? BASIC

It receives telemetry, processes it (batching, filtering, redaction, sampling) and exports it to one or more backends, so those decisions live in config instead of application code.

Q3. You must move from Jaeger to a managed backend with zero app changes. How? SCENARIO

If apps export OTLP to a collector, add the new exporter in the collector config, run both in parallel to compare, then remove the old one.

Q4. Traces show up as 'unknown_service'. Why? SCENARIO

The resource attribute service.name isn't set; set OTEL_SERVICE_NAME (and version and environment) on every service.

Q5. Is OpenTelemetry a backend? TRICKY

No: it produces, processes and transports telemetry. Storage, querying and dashboards come from Jaeger, Tempo, Prometheus or a vendor.

Q6. Why do semantic conventions matter if every service already sends data? TRICKY

Without shared names, the same field is called five things in five languages, so cross-service queries, dashboards and alerts break.

Next: SD-N-007 · Correlation and Trace IDs backendroadmap.com